MOTORFLOW

Informe Técnico: Configuración, Ejecución y Manual Completo de Pruebas Unitarias

Proyecto: Ingeniería de Software 1 (Proyecto-IS1) | **Fecha:** Febrero 2026 | **Estado:** 100% Operativo y Verificado (20/20 Pruebas Exitosas)

✓ **Estado Global:** Tanto el backend de Spring Boot, como la base de datos MySQL (`tallerdb`), el frontend interactivo y la suite de **20 pruebas unitarias con JUnit 5 (incluyendo suite de Ciberseguridad y Control de Acceso RBAC)** se encuentran completamente configurados y funcionando con un 100% de éxito (0 fallos, 0 errores).

1. Diagnóstico y Corrección en IntelliJ IDEA

Al intentar arrancar la aplicación por primera vez pulsando el botón verde de Play en `TallerApplication.java`, IntelliJ IDEA mostraba el mensaje de error: "*The file in the editor is not runnable (Mayús+F10)*".

Causa del problema:

- El proyecto abierto en IntelliJ era la carpeta raíz del repositorio (`Proyecto-IS1`), la cual contiene dos subcarpetas independientes: `backend` y `frontend`.
- IntelliJ no había detectado automáticamente el archivo `backend/pom.xml` como raíz de un módulo Maven, tratando los archivos Java como texto simple sin compilar y sin reconocer el método `main` como ejecutable.

Cambios aplicados y solución:

1. Se configuró el archivo `.idea/misc.xml` vinculando explícitamente `backend/pom.xml` a través del componente `MavenProjectsManager` y asignando el JDK 17 correspondiente.
2. Se creó la configuración del compilador en `.idea/compiler.xml` para el procesamiento de anotaciones de Spring Boot.
3. Se generó la configuración de ejecución en `.idea/runConfigurations/TallerApplication.xml`, permitiendo a IntelliJ habilitar de forma directa el botón de Play para la clase principal.

2. Verificación de la Base de Datos MySQL

Se revisó la conectividad con la base de datos configurada en `backend/src/main/resources/application.properties`:

- **Servicio activo:** El servicio de Windows `MySQL80` está en estado *Running*.
- **Base de datos:** La base de datos `tallerdb` existe en el servidor local (puerto `3306`).
- **Credenciales:** Usuario `root` y contraseña `admin123` autenticados correctamente.
- **Semilla de datos:** El componente `DatosPrueba.java` inserta de forma automática los usuarios, vehículos, materiales y reparaciones de prueba en el primer arranque.

3. Ejecución del Frontend y Conexión con el Backend

El frontend de MotorFlow está construido con HTML, CSS y JavaScript nativo, lo que permite abrirlo de forma inmediata sin necesidad de instalar dependencias de Node.js o librerías pesadas.

Cómo abrir el frontend:

- **Opción recomendada:** Abrir el archivo `frontend/index.html` en IntelliJ y hacer clic en el icono flotante del navegador (Chrome / Edge) en la esquina superior derecha del editor.
- **Alternativa:** Hacer doble clic directamente sobre `frontend/index.html` en el explorador de archivos de Windows.

Cuentas de usuario disponibles para pruebas:

Rol	Correo electrónico	Contraseña	Funcionalidad principal
Administrador	admin@taller.com	admin123	Control total: gestión de empleados, inventario de piezas, confirmación final de reparaciones y caja del taller.
Recepcionista	laura@taller.com	laura123	Recepción de coches, alta de nuevos clientes, apertura de averías y asignación inicial.
Mecánico	pedro@taller.com	pedro123	Visualización de órdenes asignadas, registro de piezas usadas del almacén y horas de mano de obra.
Cliente	juan@email.com	juan123	Seguimiento en tiempo real de su vehículo mediante línea temporal interactiva y consulta de facturas.

4. Guía Didáctica: Pruebas Unitarias y Pruebas de Seguridad en Software

> **Definición Clave:** Una **prueba unitaria** es un bloque de código automatizado que evalúa el comportamiento de la unidad lógica más pequeña e indivisible de un sistema (habitualmente una función o método de una clase) de forma **completamente aislada** del resto de la aplicación, de la red y de la base de datos física.

4.1. ¿Por qué son vitales las Pruebas Unitarias de Seguridad?

En ingeniería de software moderna, la seguridad no se prueba únicamente al final con escaneos de red o penetration testing; se implementa y valida en el código desde el nivel unitario (**Security by Design**). Las pruebas unitarias de seguridad permiten certificar:

- **Control de Acceso Basado en Roles (RBAC):** Asegurar que un usuario autenticado con un rol técnico (ej. `MECANICO`) o externo (ej. `CLIENTE`) no pueda ejecutar operaciones reservadas a la administración o recepción (ej. abrir órdenes o facturar).
- **Protección contra Suplantación y Credenciales Huérfanas:** Bloquear accesos no autorizados a registros incompletos o presenciales sin contraseña.
- **Integridad y Prevención de Fuerza Bruta en Autenticación:** Validar que credenciales inexistentes, códigos no solicitados o contraseñas débiles sean rechazados de inmediato.

4.2. El Patrón Universal de Diseño: AAA (Arrange - Act - Assert)

Fase del Patrón AAA	¿Qué se hace en esta fase?	Ejemplo de Seguridad en MotorFlow
1. Arrange (Preparar)	Se instancian los objetos, se configuran dobles simulados (Mocks) y se preparan las credenciales o roles a auditar.	`Usuario mecanico = new Usuario(..., Rol.MECANICO);` `when(usuarioRepo.findById(2L)).thenReturn(Optional.of(mecanico));`
2. Act (Actuar / Ejecutar)	Se invoca el método protegido que se pretende auditar.	`reparacionService.crearReparacion(dto);`
3. Assert (Afirmar / Verificar)	Se comprueba con aserciones que el sistema detectó la infracción y lanzó la excepción de seguridad esperada.	`assertThrows(IllegalArgumentException.class, ...);` `assertTrue(msg.contains("no tiene permisos"));`

4.3. Dobles de Prueba: Aislamiento con Mocks (Mockito)

Para probar la lógica de un servicio (como `UsuarioService`, `ClienteService` o `ReparacionService`) sin necesidad de conectarse a MySQL ni alterar datos reales, se emplean **Mocks**:

- `mock(UsuarioRepository.class)`: Crea un repositorio virtual en memoria RAM.
- `when(repositorio.findByEmail("atacante@externo.com")).thenReturn(Optional.empty())`: Simula con exactitud la respuesta del almacén ante un email inexistente.
- `ReflectionTestUtils.setField(...)`: Inyecta el mock dentro del servicio privado.

4.4. Catálogo de Aserciones Principales en JUnit 5

Aserción	Propósito en Pruebas de Negocio y Seguridad	Ejemplo
`assertEquals(esperado, actual)`	Comprueba igualdad matemática exacta o mensajes textuales de error de seguridad.	`assertEquals("Contraseña incorrecta.", ex.getMessage());`
`assertTrue(condicion)`	Verifica que una condición booleana o de contención es	`assertTrue(cliente.tieneCuenta());`

Aserción	Propósito en Pruebas de Negocio y Seguridad	Ejemplo
	estrictamente verdadera.	
`assertFalse(condicion)`	Verifica que una condición booleana es estrictamente falsa.	`assertFalse(presupuesto.estaCaducado());`
`assertNotNull(objeto)`	Garantiza que un código o token generado no es nulo.	`assertNotNull(reparacion.getLocalizador());`
`assertThrows(Excepcion.class, lambda)`	**Esencial en seguridad:** Verifica que una acción no autorizada detona el corte inmediato de la ejecución con una excepción.	`assertThrows(IllegalArgumentException.class, () -> service.crearReparacion(dto));`

5. Catálogo Completo de las 20 Pruebas Unitarias Implementadas

La suite se localiza en `backend/src/test/java/com/taller/PruebasUnitariasTest.java` y cubre la totalidad de la lógica funcional, contable y de seguridad del taller:

Nº	Clase y Método	Objetivo y Comprobación de Negocio / Seguridad	Aserciones utilizadas
1	`Factura recalcularImportes()`	Calcula con precisión materiales (100 €) y mano de obra (70 €), aplicando el margen (15% = 25,50 €), IVA (21% = 41,06 €) y total (236,56 €).	`assertEquals(esperado, actual, delta)`
2	`Factura recalcularImportes()`	Comprueba que cuando no hay materiales (coste 0 €), se calcula la factura únicamente sobre la mano de obra sin fallos matemáticos.	`assertEquals(0.0, costeMateriales)` `assertEquals(48.70, totalFactura)`
3	`PresupuestoReparacion recalcularTotales()`	Suma correctamente las líneas de presupuesto, discriminando subtotal de piezas respecto a la mano de obra.	`assertEquals(50.0, subtotalPiezas)` `assertEquals(60.0, subtotalManoObra)`
4	`PresupuestoReparacion estaCaducado()`	Verifica la lógica temporal: fechas futuras devuelven no caducado y fechas pasadas devuelven caducado.	`assertFalse(estaCaducado())` `assertTrue(estaCaducado())`
5	`Reparacion generarLocalizador()`	Comprueba que el localizador de seguimiento no es nulo y tiene una longitud exacta de 8 caracteres alfanuméricos.	`assertNotNull(localizador)` `assertEquals(8, length())`
6	`Material Constructor y Getters`	Valida la consistencia de los datos del inventario (nombre, descripción, stock físico y precio unitario).	`assertEquals("Filtro...", nombre)` `assertEquals(15, stockDisponible)`
7	`UsuarioService login()`	**Login exitoso empleado:** Verifica que un empleado con email y contraseña válidos inicia sesión y recibe su rol correspondiente.	`assertNotNull(response)` `assertEquals("MECANICO", rol)`
8	`UsuarioService login()`	**Contraseña incorrecta:** Verifica que si la contraseña es errónea, se bloquea el acceso lanzando `IllegalArgumentException`.	`assertThrows(IllegalArgumentException.class, ...)`
9	`Cliente tieneCuenta()`	Distingue correctamente si un cliente dispone de cuenta web con credenciales o es solo un cliente presencial de paso.	`assertFalse(sinCuenta)` `assertTrue(conCuenta)`
10	`CitaPrevia generarCodigoCita()`	Comprueba que el código identificativo de cita previa se genera con el prefijo oficial `CITA-` y una longitud exacta de 9 caracteres.	`assertNotNull(codigo)` `assertTrue(startsWith("CITA-"))` `assertEquals(9, length())`
11	`MaterialService actualizarStock()`	**Protección de stock negativo:** Verifica que la lógica de almacén prohíbe stocks negativos lanzando `IllegalArgumentException` con el mensaje exacto.	`assertThrows(IllegalArgumentException.class, ...)` `assertEquals("El stock no puede ser negativo.", msg)`
12	`ClienteService registrarCliente()`	**Control de duplicados:** Rechaza el registro de un cliente si el correo	`assertThrows(...)` `assertTrue(msg.contains("Ya existe..."))`

Nº	Clase y Método	Objetivo y Comprobación de Negocio / Seguridad	Aserciones utilizadas
		electrónico ya existe en la base de datos.	
13	`Reparacion` `Constructor con estimación`	Verifica que el estado inicial de una orden de trabajo es `PENDIENTE` y calcula la fecha estimada de fin sumando los días a la fecha de entrada.	`assertEquals(PENDIENTE, estado)` `assertEquals(hoy + 5, fechaEstimadaFin)`
14	`Vehiculo` `Constructor y relación`	Verifica la vinculación bidireccional y la integridad referencial entre un Vehículo (matrícula, marca, modelo, año) y su Cliente propietario.	`assertNotNull(getCliente())` `assertEquals("Ana Martínez", ...)`

15	`ConfiguracionTaller`
	`Valores por defecto`
		Comprueba la consistencia del Singleton de negocio: 15% margen comercial, 35 €/h precio de mano de obra y 2000 € de fondo de tesorería base.	`assertEquals(15.0, margen)`
			`assertEquals(35.0, precioHora)`
			`assertEquals(2000.0, fondoInicial)`

16	`UsuarioService` `login()`	**SEGURIDAD (Autenticación):** Rechazo fulminante de intento de inicio de sesión con correo inexistente en el sistema, protegiendo contra enumeración y accesos no válidos.	`assertThrows(IllegalArgumentException.class, ...)` `assertTrue(msg.contains("Usuario no encontrado"))`
17	`ReparacionService` `crearReparacion()`	**SEGURIDAD (Control de Acceso RBAC):** Bloqueo estricto de apertura de órdenes a usuarios sin rol de recepción (ej. un mecánico). Cumplimiento del principio de mínimo privilegio.	`assertThrows(IllegalArgumentException.class, ...)` `assertTrue(msg.contains("no tiene permisos de recepcionista"))`
18	`ReparacionService` `asignarMecanico()`	**SEGURIDAD (Control de Acceso RBAC):** Bloqueo de asignación técnica de vehículos a usuarios sin rol mecánico (ej. asignar el coche a un recepcionista).	`assertThrows(IllegalArgumentException.class, ...)` `assertTrue(msg.contains("no tiene rol de mecánico"))`
19	`ClienteService` `loginCliente()`	**SEGURIDAD (Protección de Cuentas):** Impide el inicio de sesión web a fichas de clientes creadas presencialmente en taller que aún no cuentan con credenciales autorizadas.	`assertThrows(IllegalArgumentException.class, ...)` `assertEquals("Este cliente no tiene cuenta...", msg)`
20	`RecuperacionPasswordService` `verificarCodigo()`	**SEGURIDAD (Flujo Antifraude):** Rechaza la verificación de códigos si no existe una petición previa activa en memoria, mitigando ataques de fuerza bruta y manipulación de tokens.	`assertThrows(IllegalArgumentException.class, ...)` `assertEquals("No hay ninguna solicitud activa...", msg)`

6. Guía de Ejecución y Validación de las Pruebas

A. Ejecución desde IntelliJ IDEA:

1. En la vista de proyecto lateral, despliega la ruta: `backend > src > test > java > com.taller`.
2. Haz clic derecho sobre el archivo **`PruebasUnitariasTest.java`**.
3. Selecciona la opción **Run 'PruebasUnitariasTest'** (o atajo de teclado Mayús+F10 / Ctrl+Shift+F10).
4. En el panel inferior de IntelliJ se mostrarán los **20 tests con el check verde de aprobación**.

B. Ejecución por consola con Maven:

Desde una terminal de PowerShell en la carpeta `backend`, ejecuta el comando de pruebas:

```
.\apache-maven-3.9.6\bin\mvn.cmd test -Dtest=PruebasUnitariasTest
```

Salida oficial obtenida en consola (100% de éxito en 1.45 s):

```
[INFO] ------------------------------------------------------- [INFO] T E S T S [INFO] ------------------------
------------------------- [INFO] Running com.taller.PruebasUnitariasTest [INFO] Tests run: 20, Failures: 0,
Errors: 0, Skipped: 0, Time elapsed: 1.454 s -- in com.taller.PruebasUnitariasTest [INFO] [INFO] Results: [INFO]
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0 [INFO] [INFO] ----------------------------------------
------------------------- [INFO] BUILD SUCCESS [INFO] ------------------------------------------------------------
------------- [INFO] Total time: 7.931 s [INFO] Finished at: 2026-11-10T10:45:42+01:00 [INFO] -------------------
------------------------------------------------
```

7. Pasos para Crear una Nueva Prueba Unitaria de Seguridad (Checklist Rápido)

1. **Identificar el vector o regla de seguridad:** Determinar qué condición de autorización o validación (rol, contraseña, token) debe proteger la operación.
2. **Configurar el escenario malicioso o no autorizado (Arrange):** Simular mediante Mocks al actor no permitido (ej. un usuario con rol inadecuado o credenciales inválidas).
3. **Ejecutar y atrapar el fallo (Act & Assert):** Utilizar `assertThrows(IllegalArgumentException.class, ...)` comprobando que el método detiene la acción sin efectos colaterales.
4. **Validar el mensaje de seguridad:** Comprobar con `assertTrue` o `assertEquals` que el mensaje de error es explícito y coherente con las políticas del taller.

Archivos entregables disponibles en la carpeta `Pruebas_unitarias/`:

- `DOCUMENTO_CAMBIOS_Y_PRUEBAS.pdf`: Informe técnico completo con la suite de 20 pruebas y módulo de seguridad.
- `DOCUMENTO_CAMBIOS_Y_PRUEBAS.html`: Versión interactiva para navegador.
- `CODIGO_PRUEBAS_UNITARIAS.txt`: Archivo de texto con el código Java íntegro de las 20 pruebas unitarias listo para copiar.